

CS 573: Homework 2

Stephan Zapodeanu

Part 1

(i)

```
In [5]: import numpy as np
import pandas as pd

def replace_quotes(data):
    replace_cnt = 0
    for i in range(len(data.loc[:, 'race'])):
        if '\\' in data.loc[:, 'race'][i]:
            replace_cnt += 1
        if '\\' in data.loc[:, 'race_o'][i]:
            replace_cnt += 1
        if '\\' in data.loc[:, 'field'][i]:
            replace_cnt += 1

    print("Quotes removed from {0} cells.".format(replace_cnt))
    data['race'] = data['race'].str.replace('\\', '')
    data['race_o'] = data['race_o'].str.replace('\\', '')
    data['field'] = data['field'].str.replace('\\', '')
    return data
```

(ii)

```
In [6]: def to_lowercase(data):
    lower_data = data['field'].str.lower()
    lower_cnt = 0
    for (lower, row) in zip(lower_data, data['field']):
        if lower != row:
            lower_cnt += 1
    print("Standardized {0} cells to lower case.".format(lower_cnt))
    data['field'] = lower_data

    return data
```

(iii)

```
In [7]: def numeric_values(data):
    RACE = 4 # column of race
    RACE_0 = 5 # column of race_o
    GENDER = 1 # column of gender
    FIELD = 9 # column of field

    data = data.sort_values(by=['race'])
```

```

race_dict = {}
cnt = 0

for row in data.itertuples():
    if row[RACE] not in race_dict:
        race_dict[row[RACE]] = cnt
        cnt += 1

data = data.replace({'race': race_dict})

data = data.sort_values(by=['race_o'])
raceo_dict = {}
cnt = 0

for row in data.itertuples():
    if row[RACE_0] not in raceo_dict:
        raceo_dict[row[RACE_0]] = cnt
        cnt += 1

data = data.replace({'race_o': raceo_dict})

data = data.sort_values(by=['gender'])
gender_dict = {}
cnt = 0

for row in data.itertuples():
    if row[GENDER] not in gender_dict:
        gender_dict[row[GENDER]] = cnt
        cnt += 1

data = data.replace({'gender': gender_dict})

data = data.sort_values(by=['field'])
field_dict = {}
cnt = 0

for row in data.itertuples():
    if row[FIELD] not in field_dict:
        field_dict[row[FIELD]] = cnt
        cnt += 1

data = data.replace({'field': field_dict})

print('Value assigned for male in column gender: {0}'.format(gender_dict['M']))
print('Value assigned for European/Caucasian-American in column race: {0}'.format(race_dict['European/Caucasian-American']))
print('Value assigned for Latino/Hispanic American in column race_o: {0}'.format(raceo_dict['Latino/Hispanic American']))
print('Value assigned for law in column field: {0}'.format(field_dict['law']))

return data

```

(iv)

```

In [9]: def normalize_cols(data):

        participant_cols = ["attractive_important", "sincere_important", "intell

```

```

        "funny_important", "ambition_important", "shared_interests_important"]

partner_cols = ["pref_o_attractive", "pref_o_sincere", "pref_o_intelligence",
                "pref_o_ambitious", "pref_o_shared_interests"]

i = 0
for index, row in data.iterrows():
    participant_total = 0
    partner_total = 0
    for col in participant_cols:
        participant_total += row[col]
    for col in partner_cols:
        partner_total += row[col]

    for col in participant_cols:
        data.at[i, col] = row[col] / participant_total
    for col in partner_cols:
        data.at[i, col] = row[col] / partner_total
    i += 1
return data

# Function that prints the means of the columns normalized above
def print_means(data):
    cols = ["attractive_important", "sincere_important", "intelligence_important",
            "funny_important", "ambition_important", "shared_interests_important",
            "pref_o_attractive", "pref_o_sincere", "pref_o_intelligence", "pref_o_ambitious",
            "pref_o_shared_interests"]

    for col in cols:
        print("Mean of {0}: {1:.2f}.".format(col, np.mean(data[col])))

data_1 = pd.read_csv('./dating-full.csv')
data_1 = replace_quotes(data=data_1)
data_1 = to_lowercase(data=data_1)
data_1 = numeric_values(data=data_1)
data_1 = normalize_cols(data=data_1)
print_means(data=data_1)
data_1.to_csv('./dating.csv')

```

Quotes removed from 8316 cells.

Standardized 5707 cells to lower case.

Value assigned for male in column gender: 1

Value assigned for European/Caucasian-American in column race: 2.

Value assigned for Latino/Hispanic American in column race_o: 3.

Value assigned for law in column field: 121

Mean of attractive_important: 0.22.

Mean of sincere_important: 0.17.

Mean of intelligence_important: 0.20.

Mean of funny_important: 0.17.

Mean of ambition_important: 0.11.

Mean of shared_interests_important: 0.12.

Mean of pref_o_attractive: 0.22.

Mean of pref_o_sincere: 0.17.

Mean of pref_o_intelligence: 0.20.

Mean of pref_o_funny: 0.17.

Mean of pref_o_ambitious: 0.11.

Mean of pref_o_shared_interests: 0.12.

Part 2

(i)

```
In [10]: import matplotlib.pyplot as plt

def visualize_gender(data):
    grouped = data.groupby('gender')
    females = grouped.get_group(0)
    males = grouped.get_group(1)
    barWidth = .3
    cols = ["attractive_important", "sincere_important", "intelligence_important",
            "funny_important", "ambition_important", "shared_interests_important"]
    male_means = []
    female_means = []

    for col in cols:
        male_means.append(np.mean(males[col]))
        female_means.append(np.mean(females[col]))

    fig = plt.subplots(figsize= (12, 8))

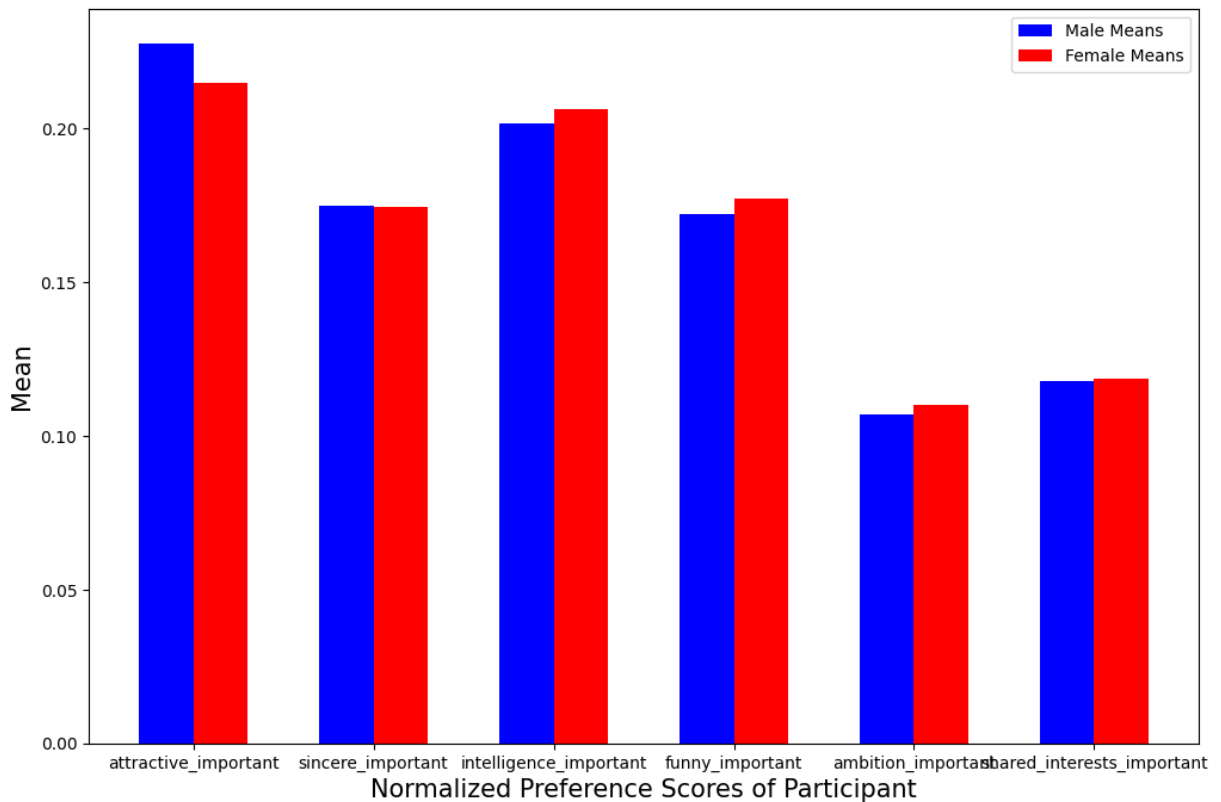
    br1 = np.arange(len(cols))
    br2 = [x + barWidth for x in br1]

    plt.bar(br1, male_means, color='blue', width=barWidth, label='Male Means')
    plt.bar(br2, female_means, color='red', width=barWidth, label='Female Means')

    plt.xlabel('Normalized Preference Scores of Participant', fontsize=15)
    plt.ylabel('Mean', fontsize=15)
    plt.xticks([r + barWidth - .15 for r in range(len(cols))], cols)

    plt.legend()
    plt.show()

data_2 = pd.read_csv('./dating.csv')
visualize_gender(data=data_2)
```



Although there are similarities between males and females on what they prefer, there are also some differences. They both seem to prefer attractiveness the most, and then intelligence the second most. Males seem to place more emphasis on attractiveness than females, and females seem to place more emphasis on intelligence, funny, and ambition. It is interesting to note that both males and females seem to value the sincerity of the other person the same amount. Overall, it appears that males and females seem to be pretty similar in what they favor within their partners.

(ii)

```
In [13]: def scatter_plots(data):
    cols = ['attractive_partner', 'sincere_partner', 'intelligence_partner',
            'funny_partner', 'ambition_partner', 'shared_interests_partner']

    distinct_vals = {}

    for col in cols:
        distinct_vals[col] = data[col].unique()

    second_date_percent = []
    for col in cols:
        temp = {}
        for val in distinct_vals[col]:
            matching = data.loc[data[col] == val]
            cnt = matching['decision'].value_counts()
            cnt = dict(cnt)
            if 1 not in cnt.keys():
                temp[val] = 0
```

```

elif 0 not in cnt.keys():
    temp[val] = 1
else:
    temp[val] = cnt[1]/(cnt[0] + cnt[1])
second_date_percent.append(temp)

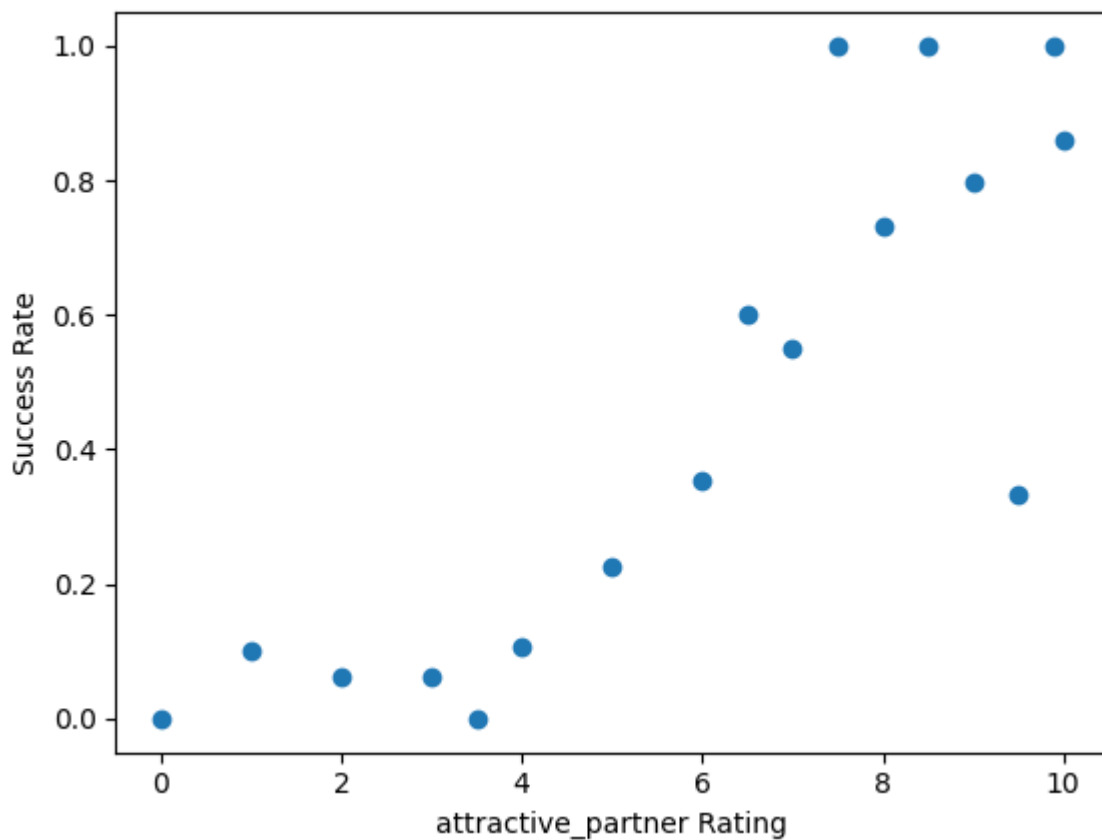
#print(second_date_percent)

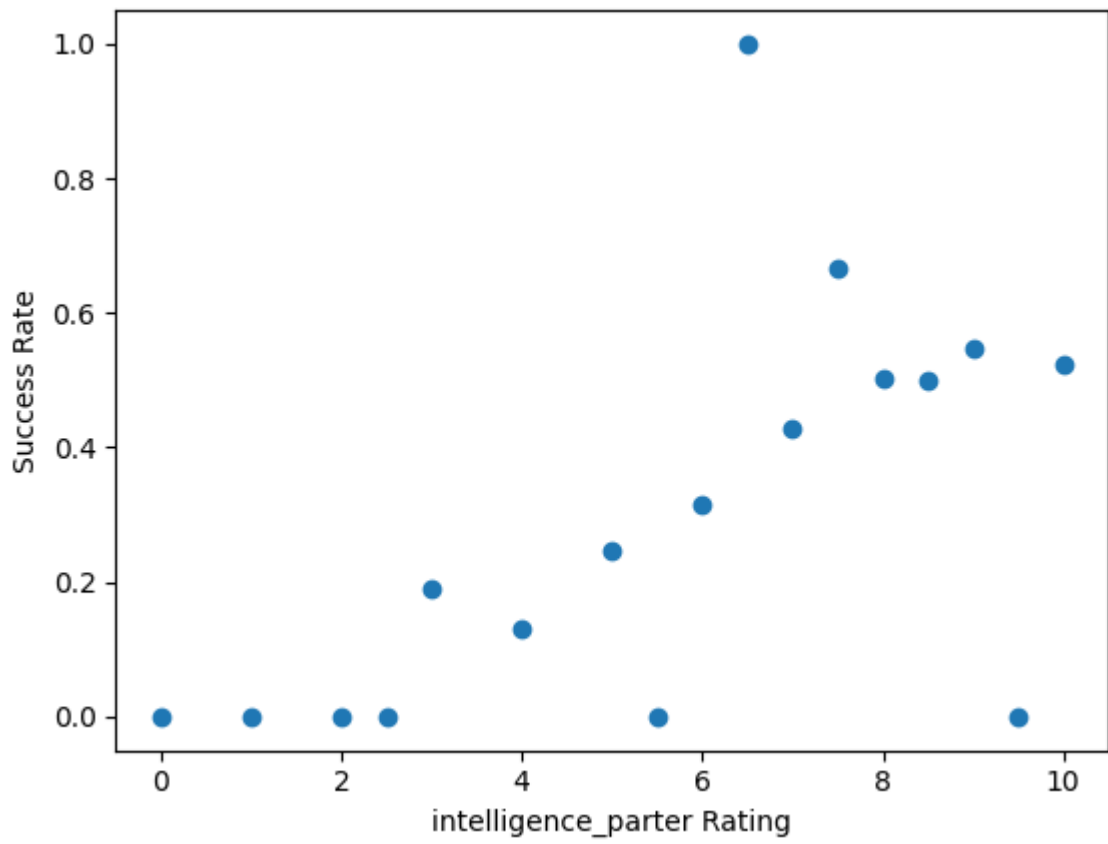
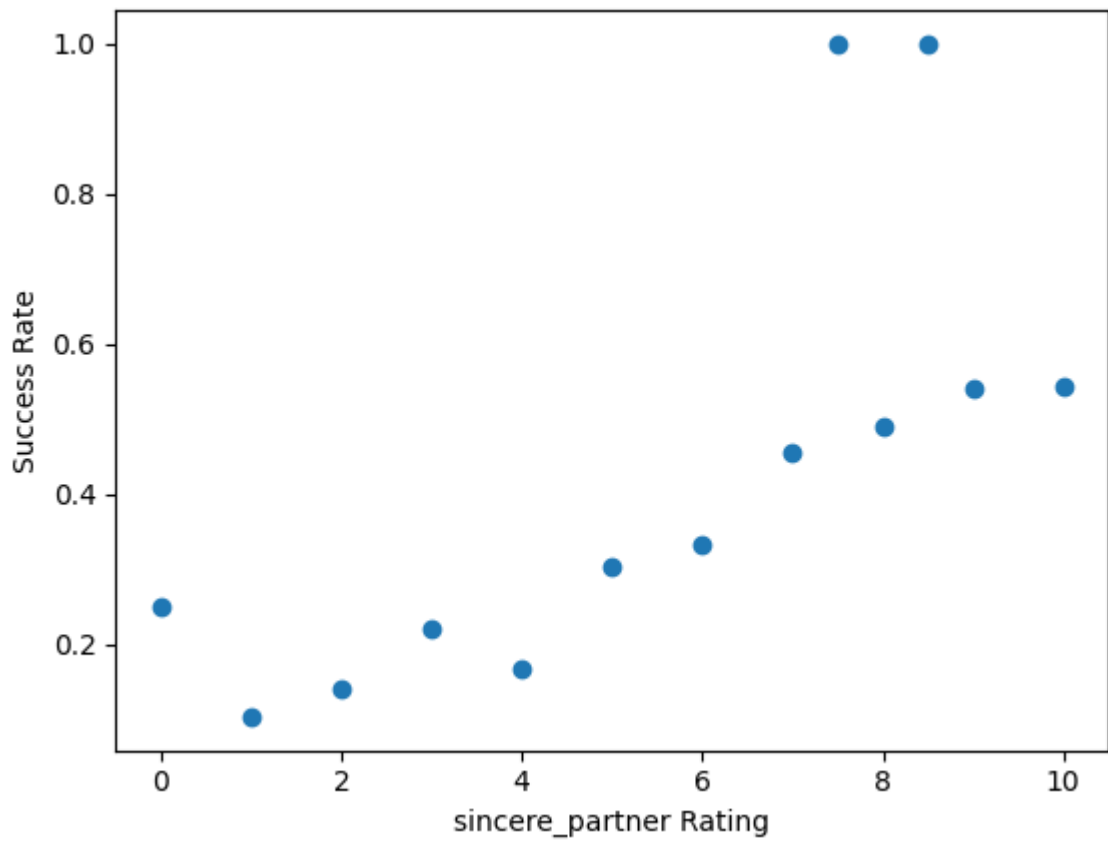
for col in range(len(cols)):
    #print(second_date_percent[col].keys())
    x = second_date_percent[col].keys()
    y = second_date_percent[col].values()
    plt.figure(cols[col])
    plt.scatter(x, y)
    plt.xlabel('{0} Rating'.format(cols[col]))
    plt.ylabel('Success Rate')

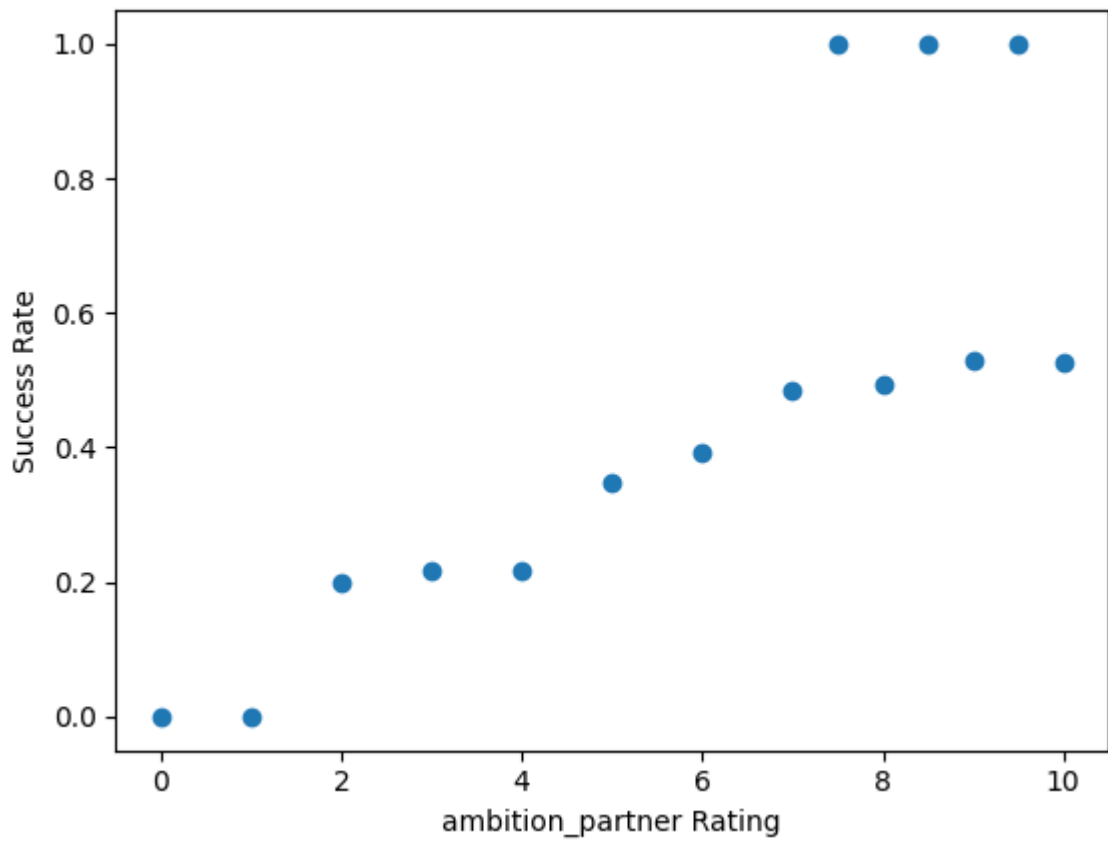
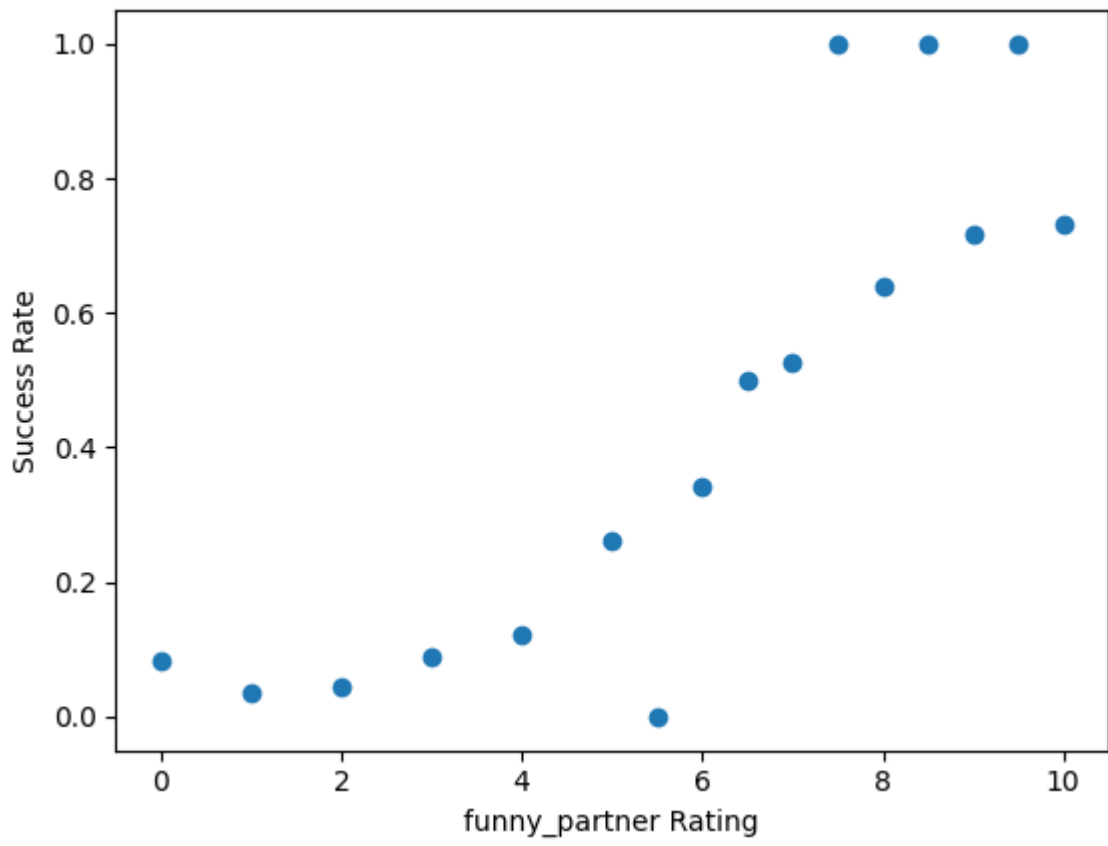
    plt.show()

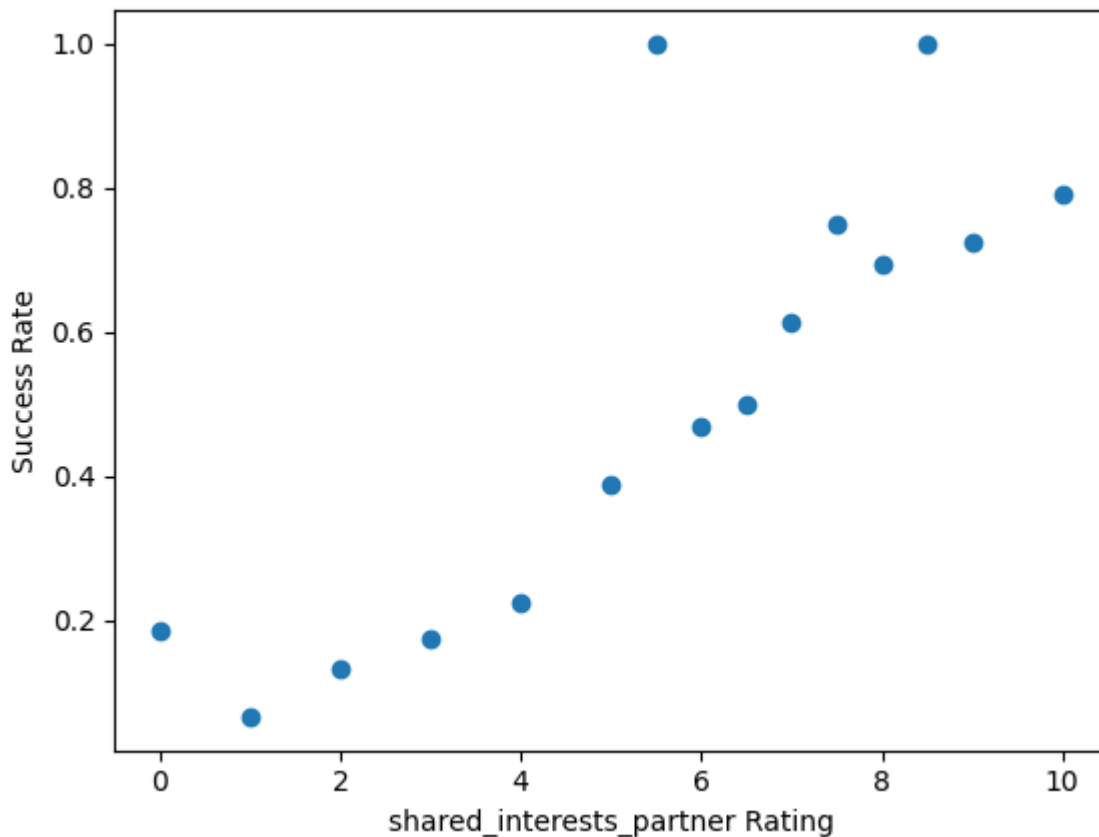
```

```
scatter_plots(data=data_2)
```









Every scatter plot shows that there is a clear correlation between the partner rating and the success rate, although some are more extreme than others. For shared_interests, attractive, and funny partner ratings, if they are close to 10, there is nearly a success rate of 100%. This can be seen most clearly in attractive_partner. Ambition, Intelligence, and Sincere still have a positive correlation but it is not as extreme, outside of some ratings that are outliers. These categories all stay at around 50% for when the rating is close to 10. One interesting observation, is that every scatter plot shows some sort of plateau (the slope gets closer to 0) as around the 8 rating mark. Overall, there is the same clear trend: The higher the rating, the higher the success rate is.

Part 3

(i)

```
In [18]: import pandas as pd
import numpy as np

ages = ['age', 'age_o']
zero_to_ten = ['important_same_race', 'important_same_religion', 'attractive_partner', 'ambition', 'attractive_partner', 'sincere_partner', 'intelligence', 'ambition_partner', 'shared_interests_partner', 'sports', 'tv', 'hiking', 'gaming', 'clubbing', 'reading', 'tv', 'theater', 'shopping', 'yoga', 'expected_happy_with_sd_people', 'like']

neg_to_pos_one = ['interests_correlate']
```

```

def set_max(data):
    for col in list(data.columns):
        if col in ages:
            data.loc[:, col].clip(lower=18, upper=58, inplace=True)
        if col in zero_to_ten:
            data.loc[:, col].clip(lower=0, upper=10, inplace=True)
        if col in neg_to_pos_one:
            data.loc[:, col].clip(lower=-1, upper=1, inplace=True)

    return data

def into_bins(data, bins, file_name):
    new_data = data.copy()
    #new_data.drop(labels='Unnamed: 0', axis=1, inplace=True)
    skip_cols = ['gender', 'race', 'race_o', 'samerace', 'field', 'decision']
    cols = list(new_data.columns)
    for col in cols:
        if col not in skip_cols:
            new_data[col], interval = pd.cut(x=data[col], bins=bins, right=True)
            counts = new_data[col].value_counts(sort=False)
            # need to uncomment this line
            print('{0}: {1}'.format(col, counts.values))

    new_data.to_csv(file_name, index=False)

data_3 = pd.read_csv('./dating.csv')
data_3 = set_max(data=data_3)
into_bins(data=data_3, bins=5, file_name='./dating-binned.csv')

```

```

age: [3032 3390 297 20 5]
age_o: [2990 3362 371 16 5]
importance_same_race: [2980 1213 977 1013 561]
importance_same_religion: [3203 1188 1110 742 501]
pref_o_attractive: [4333 1987 344 51 29]
pref_o_sincere: [1416 4378 865 79 6]
pref_o_intelligence: [666 3935 1873 189 81]
pref_o_funny: [1255 4361 1048 55 25]
pref_o_ambitious: [1963 2352 2365 42 22]
pref_o_shared_interests: [1506 2068 1981 1042 147]
attractive_important: [4323 2017 328 57 19]
sincere_important: [546 2954 2782 377 85]
intelligence_important: [630 3976 1861 210 67]
funny_important: [1282 4306 1070 58 28]
ambition_important: [1913 2373 2388 49 21]
shared_interests_important: [1464 2197 1950 1007 126]
attractive: [131 726 899 4122 866]
sincere: [57 228 352 2715 3392]
intelligence: [127 409 732 3190 2286]
funny: [19 74 1000 2338 3313]
ambition: [225 697 559 2876 2387]
attractive_partner: [284 948 2418 2390 704]
sincere_partner: [94 353 1627 3282 1388]
intelligence_partner: [36 193 1509 3509 1497]
funny_partner: [279 733 2296 2600 836]
ambition_partner: [119 473 2258 2804 1090]
shared_interests_partner: [701 1269 2536 1774 464]
sports: [650 961 1369 2077 1687]
tv_sports: [2151 1292 1233 1383 685]
exercise: [619 952 1775 2115 1283]
dining: [39 172 1118 2797 2618]
museums: [117 732 1417 2737 1741]
art: [224 946 1557 2500 1517]
hiking: [963 1386 1575 1855 965]
gaming: [2565 1522 1435 979 243]
clubbing: [912 1068 1668 2193 903]
reading: [131 398 1071 2317 2827]
tv: [1188 1216 1999 1642 699]
theater: [288 811 1585 2300 1760]
movies: [144 462 530 2783 2825]
concerts: [222 777 1752 2282 1711]
music: [62 196 1106 2583 2797]
shopping: [1093 1098 1709 1643 1201]
yoga: [2285 1392 1369 1056 642]
interests_correlate: [75 985 2312 2597 775]
expected_happy_with_sd_people: [321 1262 3292 1596 273]
like: [273 865 2539 2560 507]

```

Part 4

```

In [19]: def split(data, test_file, training_file):
          test_data = data.sample(frac=0.2, random_state=47)
          test_data.to_csv(test_file, index=False)
          training_data = data.drop(test_data.index)
          training_data.to_csv(training_file, index=False)

```

```
data_bined = pd.read_csv('./dating-binned.csv')
split(data=data_bined, test_file='./testSet.csv', training_file='./trainingS
```

Part 5

(i)

```
In [29]: class NBC:
    def __init__(self, t_frac, laplace=1):
        self.t_frac = t_frac
        self.laplace = laplace

    def nbc(self, t_frac):
        binned_data = pd.read_csv('./dating-binned.csv')
        binned_data.drop('Unnamed: 0', axis = 1, inplace=True)

        data = pd.read_csv('./trainingSet.csv')
        data.drop('Unnamed: 0', axis=1, inplace=True)
        trainingData = data.sample(random_state=47, frac=t_frac)

        class_probs = self.classProbs(trainingData)
        probs_dict = self.getLabelProbs(training_data=trainingData, overall_
        training_percentage = self.predict_decisions(data=trainingData, probs=
        print('Training Accuracy: {0:.2f}'.format(training_percentage))

        testData = pd.read_csv('./testSet.csv')
        testData.drop('Unnamed: 0', axis=1, inplace=True)
        testingData = testData.sample(random_state=47, frac=t_frac)

        testing_percentage = self.predict_decisions(data=testingData, probs=
        print('Testing Accuracy: {0:.2f}'.format(testing_percentage))
        return (training_percentage, testing_percentage)

    def classProbs(self, trainingData):
        class_probs = {}
        for val in trainingData['decision']:
            if val not in class_probs:
                class_probs[val] = 1
            else:
                class_probs[val] += 1

        class_probs[0] = class_probs[0] / (class_probs[0] + class_probs[1])
        class_probs[1] = 1 - class_probs[0]
        return class_probs

    def getLabelProbs(self, training_data, overall_data):
        decision_data = training_data['decision']
        probs = {}

        for col in list(training_data.columns):
```

```

        probs[col] = self.labelProb(label_data=training_data[col], class

    return probs

def labelProb(self, label_data, class_probs, overall_data):
    label_prob = {}
    # Look at the entire dataset for every possible label
    for val in overall_data:
        if val not in label_prob:
            label_prob[val] = 0

    probs = {}
    for val in label_prob:
        for cat in set(class_probs):
            val_count = 0
            cat_count = 0
            for i, j in zip(label_data, class_probs):
                if i == val and j == cat:
                    val_count += 1
                if j == cat:
                    cat_count += 1
            # Smoothing
            probs[(val, cat)] = (val_count + self.laplace)/(cat_count +

    return probs

def predict_decisions(self, data, probs, class_probs):
    correct, total = 0, 0
    for index, row in data.iterrows():
        one_prob = class_probs[1]
        zero_prob = class_probs[0]
        for col in data.columns:
            if col != 'decision':
                one_prob *= probs[col][(row[col], 1)]
                zero_prob *= probs[col][(row[col], 0)]

        if one_prob > zero_prob:
            if row['decision'] == 1:
                correct += 1
        elif one_prob < zero_prob:
            if row['decision'] == 0:
                correct += 1
        else:
            correct += 1

        total += 1
    return correct / total

classifier = NBC(t_frac=1, laplace=1)
t1, t2 = classifier.nbc(t_frac=classifier.t_frac)

```

Training Accuracy: 0.77

Testing Accuracy: 0.74

(ii)

```
In [35]: def into_bins_2(data, bins, file_name):
    new_data = data.copy()
    #new_data.drop(labels='Unnamed: 0', axis=1, inplace=True)
    skip_cols = ['gender', 'race', 'race_o', 'samerace', 'field', 'decision']
    cols = list(new_data.columns)
    for col in cols:
        if col not in skip_cols:
            new_data[col], interval = pd.cut(x=data[col], bins=bins, right=True)
            counts = new_data[col].value_counts(sort=False)
            # Line commented for part 5 section 2
            #print('{0}: {1}'.format(col, counts.values))

    new_data.to_csv(file_name, index=False)

def run_bins():
    bins = [2, 5, 10, 50, 100, 200]
    training_percentages = []
    testing_percentages = []
    for bin_ in bins:
        data = pd.read_csv('./dating.csv')
        data = set_max(data)
        into_bins_2(data=data, bins=bin_, file_name='./dating-binned-2.csv')
        data_binned = pd.read_csv('./dating-binned-2.csv')
        data_binned.drop('Unnamed: 0', inplace=True, axis=1)
        split(data=data_binned, test_file='./testSet-binned.csv', training_file='./trainingSet-binned.csv')
        print('Bin size: {0}'.format(bin_))
        classifier = NBC(t_frac=1, laplace=1)

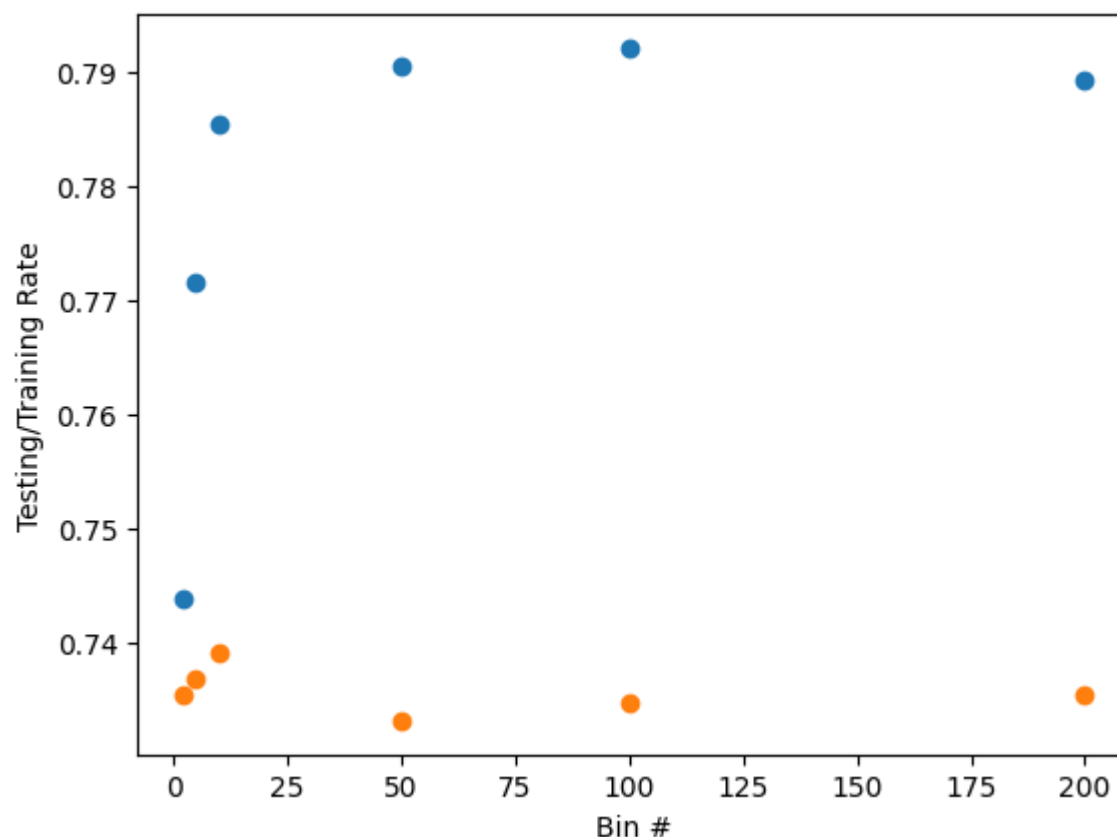
        data = pd.read_csv('./trainingSet-binned.csv')
        trainingData = data.sample(random_state=47, frac=classifier.t_frac)
        class_probs = classifier.classProbs(trainingData)
        probs_dict = classifier.getLabelProbs(training_data=trainingData, overfitting=0)
        percentage = classifier.predict_decisions(data=trainingData, probs=probs_dict)
        print('Training Accuracy: {0:.2f}'.format(percentage))
        training_percentages.append((percentage))

        data = pd.read_csv('./testSet-binned.csv')
        testData = data.sample(random_state=47, frac=classifier.t_frac)
        percentage = classifier.predict_decisions(data=testData, probs=probs_dict)
        print('Testing Accuracy: {0:.2f}'.format(percentage))
        testing_percentages.append(percentage)

    plt.scatter(bins, training_percentages)
    plt.scatter(bins, testing_percentages)
    plt.xlabel('Bin #')
    plt.ylabel('Testing/Training Rate')
    plt.show()

run_bins()
```

Bin size: 2
 Training Accuracy: 0.74
 Testing Accuracy: 0.74
 Bin size: 5
 Training Accuracy: 0.77
 Testing Accuracy: 0.74
 Bin size: 10
 Training Accuracy: 0.79
 Testing Accuracy: 0.74
 Bin size: 50
 Training Accuracy: 0.79
 Testing Accuracy: 0.73
 Bin size: 100
 Training Accuracy: 0.79
 Testing Accuracy: 0.73
 Bin size: 200
 Training Accuracy: 0.79
 Testing Accuracy: 0.74



As the number of bins increases, the difference between the training set and testing set increases rapidly. The only time they are similar, is when bin=2, which essentially means that each feature in the training set is split into 2. This obviously will not work very well. It's interesting to note that once the bin size > 10, they are all essentially the same error rate. Even though 100 has the highest raining percentage, it doesn't mean that it should be used because as the bins increase, so does the runtime so for extremely large datasets it will take too long. It's also interesting to note that all of hte testing rates are hovering around 73-74%. For this dataset, there is noreal difference between those, and actually as can be seen in the graph bin=10 produces the highest testing rate.

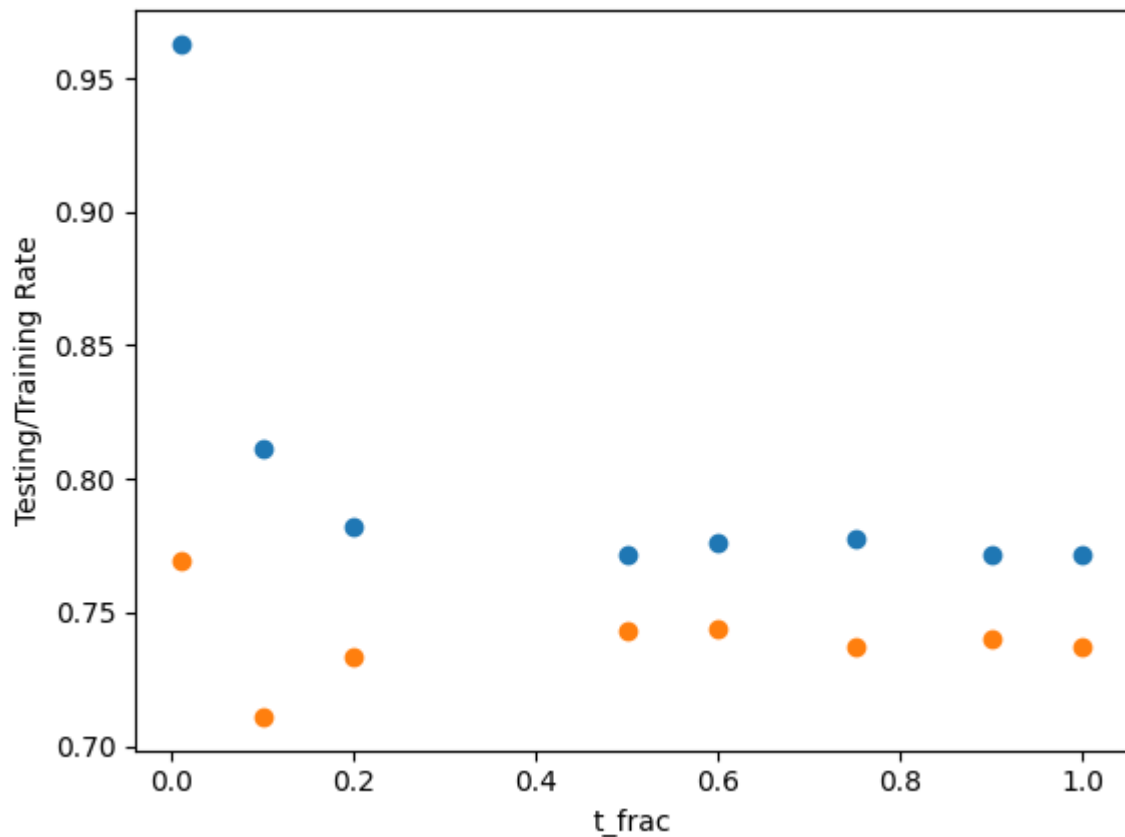
(iii)

```
In [38]: def run_fracs():
        fracs = [.01, .1, .2, .5, .6, .75, .9, 1]
        training_percentages = []
        testing_percentages = []
        for frac in fracs:
            classifier = NBC(t_frac=frac, laplace=1)
            training, testing = classifier.nbc(t_frac=classifier.t_frac)
            training_percentages.append(training)
            testing_percentages.append(testing)

        plt.scatter(fracs, training_percentages)
        plt.scatter(fracs, testing_percentages)
        plt.xlabel('t_frac')
        plt.ylabel('Testing/Training Rate')
        plt.show()

run_fracs()
```

```
Training Accuracy: 0.96
Testing Accuracy: 0.77
Training Accuracy: 0.81
Testing Accuracy: 0.71
Training Accuracy: 0.78
Testing Accuracy: 0.73
Training Accuracy: 0.77
Testing Accuracy: 0.74
Training Accuracy: 0.78
Testing Accuracy: 0.74
Training Accuracy: 0.78
Testing Accuracy: 0.74
Training Accuracy: 0.77
Testing Accuracy: 0.74
Training Accuracy: 0.77
Testing Accuracy: 0.74
```



The blue dots represent the Training accuracy and the orange dots represent the Testing Accuracy.

As the frac gets closer to 1, it appears that the Training accuracy decreases. Interestingly, the fraction with the highest testing percentage is when $\text{frac} == .01$. This is probably by chance only because of how the dataset was sampled and isn't likely to actually be the best training set. The fraction that produces the smallest training and testing set delta, is actually .5. That is also when the highest testing set percentage when there are enough samples in the training set to suggest that it is viable. One interesting aspect is how an increase in samples taken does not mean that the training/testing percentage increases. I believe this is cause there is more diversity in each sample so that outliers can't dominate as much.